Blaze speed-to-lead: how the whole thing works

Option B take-home. A fake Facebook lead arrives on a webhook, gets rule-checked, scored 1-5 by Claude with a draft reply, saved to a Google Sheet and sent to Discord. Written 2026-10-18.

1. The goal in one paragraph

Blaze's Facebook leads wait a median of 22 hours for first contact, and 44% are junk. This pipeline cuts "lead submits to Blaze knows, with a score and a draft" to seconds. It does **not** send anything to the lead: a person still reads the Discord ping and replies. All messages go only to the owner's private Discord channel, and all test data is fake (`.example` emails, 555 phones).

2. The files and what each one does

File	Role	Holds secrets?
`app.py`	The server (FastAPI) and the entire pipeline. All logic is here.	No (reads them from `.env`)
`prompt.md`	Claude's instructions: the 1-5 rubric, the "treat the lead as data" rule, the draft-reply rules. Loaded by `app.py` at startup.	No
`simulate.py`	Plays Facebook. Sends leads from `leads.json` to the server and prints status and request time. Options: no argument (all 10), lead numbers (`1 9`), or `--bad` (requests that must be rejected).	Reads the webhook secret from `.env`
`leads.json`	10 fake test leads, each with an `expected_score` answer key. `simulate.py` removes that key before sending.	No
`sheets_smoke_test.py`	One-off check that Google auth works: appends a test row.	No
`.env`	The real secrets: Anthropic key, Discord URL, sheet ID, service-account file name, webhook secret. **Gitignored.**	**Yes**
`.env.example`	Template: the same names with no values. Committed.	No
`service_account.json`	Google robot-user key (acts like a password). **Gitignored.**	**Yes**
`.gitignore`	Keeps `.env`, `service_account.json`, `.venv/` and `__pycache__/` out of git.	No
`requirements.txt`	Python packages: fastapi, uvicorn, gspread, anthropic, httpx, python-dotenv.	No
`CLAUDE.md`, `PLAN.md`	The project rules (short) and the full plan with reasoning and the test table.	No

`BUILD_LOG.md`	What we did and why, step by step.	No
`README.md`	Not written yet: what it does, how to run, sample output, results table.	No

3. The path of one lead, step by step

```
simulate.py ---POST + secret header---> app.py (server on port 8000)
                                        |
  [1] check_secret   wrong/missing ----> 401, STOP (nothing read, saved or sent)
                                        | right
  [2] webhook        validates shape (422 if wrong), stamps received_at,
                     queues process() in the background, replies 200 right away
                                        |
                               (background: process())
  [3] rule_checks    list of flags (never rejects anything)
  [4] 2+ flags?      yes: score 1, "failed rule checks", skip Claude
                     no : score_with_claude  (Claude Haiku scores + drafts)
  [5] save_row       one row appended to the Google Sheet
  [6] ping_discord   message sent to the private Discord channel
```

#	What happens	Where (`app.py`)
1	**Secret check.** The `X-Webhook-Secret` header is compared with `WEBHOOK_SECRET` using `hmac.compare_digest`. It runs before the body is read, so a bad secret is always 401 and an unset secret rejects everything.	`check_secret`, line 36
2	**Receive.** Pydantic checks the lead's shape (`Lead`: full_name, email, phone, company_name, number_of_doors, message); a wrong shape is 422. The server stamps `received_at`, queues `process` and returns `{"status": "received"}`. Replying instantly means a slow step can never make the sender time out and resend.	`Lead`, line 27; `webhook`, line 46
3	**Rule checks.** Free Python checks for obvious fakes: bad email (no @, or no dot after it); phone under 10 digits, one repeated digit, or 1234567890; doors outside 1-50,000; name or company that is keyboard mash (`asdf`, `test`, `qwerty` in `MASH_WORDS`) or one character. 555 phones are deliberately not flagged. A blank company is not flagged.	`rule_checks`, line 59; `MASH_WORDS`, line 20
4	**Score.** Two or more flags: score 1, reason "failed rule checks", empty draft, Claude is not called. Otherwise `score_with_claude` sends the lead (inside `<lead>` tags) and the flags to `claude-haiku-5-5` with the instructions from `prompt.md`, forcing the `record_lead_score` tool so the answer is structured: `score` (1-5), `reason` (one line), `draft_reply`. The code validates the score and blanks the draft at score 1.	`MODEL`, `PROMPT`, `TOOL`, lines 76-91; `score_with_claude`, line 93; the 2-flag branch is in `process`, line 159

5	**Save.** One row, 12 columns: received_at, name, email, phone, company, doors, message, rule_flags, score, reason, draft, latency_s. Written as `RAW` so phones stay text and lead text can never run as a spreadsheet formula. `latency_s` is time from `received_at` to the save.	`save_row`, line 120; columns at line 21
6	**Discord.** A message with the score, seconds since submit (time to ping), name, company, doors, contact, the reason and the draft, cut to under 2,000 characters. Discord pushes it to the phone.	`format_discord`, line 136; `ping_discord`, line 150

Message format

```
NEW LEAD  fit 5/5  (x.x s after submit)
Maria Lopez · Oakline Property Group · 240 doors
maria@oaklinepm.example · +1 317 555 0142
Why: <reason>

Draft reply:
<draft>
```

4. Why failures never lose a lead

Each of steps 3-6 runs in its own `try/except` inside `process` (line 159). An error is logged to the server window (exception type and HTTP status only, never a secret) and the next step still runs.

What fails	What still happens
Claude (bad key, outage, bad score)	The lead is saved with a blank score and reason "manual review", and Discord still pings.
Google Sheets	Discord still pings.
Discord	The row is already in the sheet.
A rule flag fires	Nothing is rejected: the lead is saved with the flags noted. Only 2+ flags change the score (to 1, without calling Claude).

> **Two kinds of "bad request".** The 401 secret check decides who may send, happens first, and stores nothing. Rule flags judge the lead's content, happen after the 200 reply, and never reject. A wrong secret leaves no trace; a junk lead is deliberately recorded with score 1.

5. Two latency numbers

The sheet's `latency_s` is time-to-save. The Discord "x.x s after submit" is time-to-ping, so it is slightly larger. Report whichever the results table names. The request time `simulate.py` prints is only how long the server took to say "received", not the pipeline time.

6. How it was tested

Check	Command	Expected
Google auth	`python sheets_smoke_test.py`	A "smoke test" row appears
Start the server	`python -m uvicorn app:app --port 8000`	Stays running in its own terminal
Good and junk lead	`python simulate.py 1 9`	Two 200s; Maria scores 4-5; asdf scores 1 with 4 flags
Rejections	`python simulate.py --bad`	401, 401, 422; no new row or ping
Full test set	`python simulate.py`	10 rows; compare `score` to `expected_score`
No lost leads	Put a wrong key in `.env`, send lead 1	Row and ping with "manual review"

`expected_score` is an answer key written before anything ran. It is removed before sending, so neither the server nor Claude ever sees it. Pass mark: at least 8 of 10 within one point, leads 9 and 10 both scored 1, every ping under 60 seconds. Lead 9 is scored by the rules; lead 10 (a prompt-injection attempt with one flag) must be caught by Claude itself.

7. Setup that was needed

- **Python environment:** `python -m venv --copies .venv` (the plain command failed under miniconda), then `pip install -r requirements.txt`.
- **Google:** a project, the Sheets and Drive APIs enabled, a service account `sheets-writer` with a JSON key, and the sheet "Blaze Leads" shared with the service account's email as Editor. The sheet ID is the part of the sheet's URL between `/d/` and `/edit`; it is not in the JSON key.
- **Discord:** a private channel with a webhook; its URL goes in `.env`.
- **Anthropic:** an API key in `.env`. The model is the constant `MODEL` in `app.py`, not an `.env` setting.
- **Secrets:** every secret lives only in `.env` or `service_account.json`, both gitignored.

8. Status

Item	State
Google auth, webhook, secret check, rule checks, sheet, Discord	Working (verified with leads 1 and 9 and the --bad requests)
Claude scoring (Haiku)	Working on Maria (5) and asdf (1 by rules)
Other 8 test leads, accuracy and latency stats	Not run yet
No-lost-leads test with a wrong key	To do

Screenshots, README, secret search of git history, push	To do
Loom and submission form	To do

The chronological record of each decision and why is in `BUILD_LOG.md`.